

Module 2 – Introduction to Programming

LAB EXERCISES

Core Lab Exercises

1. Overview of C Programming

LAB EXERCISE:

- Research and provide three real-world applications where C programming is extensively used, such as in embedded systems, operating systems, or game development.

ANS: : Real-World Applications:

1. **Operating Systems:** Windows, macOS, and the Linux kernel are largely written in C.
2. **Embedded Systems:** Used in microcontrollers for cars, washing machines, and medical devices.

Game Engines: Many graphics libraries and engines (like the foundation of Doom or Quake) use C for performance

2. Setting Up Environment

LAB EXERCISE:

- Install a C compiler on your system and configure the IDE. Write your first program to print "Hello, World!" and run it.

3. Basic Structure of a C Program

LAB EXERCISE:

- Write a C program that includes variables, constants, and comments. Declare and use different data types (int, char, float) and display their values.

ANS:

```
#include <stdio.h>
```

```
int main() {  
    const int i=10; //constant keeps the value unchangeable  
    float j=2.5;  
    char k='h';
```

```

printf("\nValue of i:%d",i);
printf("\nValue of j:%.2f",j);
printf("\nValue of char:%c",k);

return 0;
}

```

4. Operators in C

LAB EXERCISE:

- Write a C program that accepts two integers from the user and performs arithmetic, relational, and logical operations on them. Display the results.

ANS:

```

#include<stdio.h>
#include<math.h>
int main(){
    float i,j;
    printf("\nEnter the value for i and j:");
    scanf("%f %f",&i, &j);

    //Arithmetic
    printf("Addition:%f + %f = %.2f\n",i,j,i+j);
    printf("Substraction:%f - %f = %.2f\n",i,j,i-j);
    printf("Multiplication:%f * %f = %.2f\n",i,j,i*j);
    printf("Division:%f / %f = %.2f\n",i,j,i/j);
    printf("Modulo:%f %% %f = %.2f\n",i,j,fmod(i,j));

    //Relational
    printf("\nEqual to:%d",i==j);
    printf("\nNot Equal to:%d",i!=j);
    printf("\nGreater than:%d",i>j);
    printf("\nLess than:%d",i<j);
    printf("\nGreater than equal to:%d",i>=j);
    printf("\nLess than equal to:%d\n",i<=j);

    //Logical
    printf("\nAnd operator:%d",(i==j)&&(j<i));
    printf("\nOr operator:%d",(i==j)||(j<i));
    printf("\nNot operator:%d",!(i>j));

    return 0;
}

```

5. Control Flow Statements in C

LAB EXERCISE:

- Write a C program to check if a number is even or odd using an if-else statement. Extend the program using a switch statement to display the month name based on the user's input (1 for January, 2 for February, etc.).

```
#include<stdio.h>
int main(){
    int i;
    printf("\nEnter a number:");
    scanf("%d",&i);

    if(i%2==0){
        printf("\nNumber is even.\n");
    }
    else {
        printf("\nNumber is odd.\n");
    }

    switch(i){
        case 1:
            printf("\nMonth is January.");
            break;

        case 2:
            printf("\nMonth is February.");
            break;

        case 3:
            printf("\nMonth is March.");
            break;

        case 4:
            printf("\nMonth is April.");
            break;

        case 5:
            printf("\nMonth is May.");
            break;

        case 6:
```

```

        printf("\nMonth is June.");
        break;

    case 7:
        printf("\nMonth is July.");
        break;

    case 8:
        printf("\nMonth is August.");
        break;

    case 9:
        printf("\nMonth is September.");
        break;

    case 10:
        printf("\nMonth is October.");
        break;

    case 11:
        printf("\nMonth is November.");
        break;

    case 12:
        printf("\nMonth is December.");
        break;

    default:
        printf("\nWrong Input.");
    }

    return 0;
}

```

6. Looping in C

LAB EXERCISE:

- Write a C program to print numbers from 1 to 10 using all three types of loops (while, for, do-while).

```

#include<stdio.h>
int main(){

```

```

int i,j=0,k=0;
    for(i=1;i<11;i++){
        printf("%d\t",i);
    }
    printf("\n");

    while(j<10){
        j++;
        printf("%d\t",j);
    }
    printf("\n");

    do{
        k++;
        printf("%d\t",k);
    }
    while(k<10);
}

```

7. Loop Control Statements

LAB EXERCISE:

- Write a C program that uses the break statement to stop printing numbers when it reaches 5. Modify the program to skip printing the number 3 using the continue statement.

```

#include<stdio.h>
int main(){
    int i;
    for(i=0;i<10;i++){
        if(i==5){
            break;
        }
        if(i==3){
            continue;
        }
        printf("%d\t",i);
    }
}

```

8. Functions in C

LAB EXERCISE:

- Write a C program that calculates the factorial of a number using a function. Include function declaration, definition, and call.

```
#include<stdio.h>
int factorial(){
    int i,fact=1,n;
    printf("\nEnter your number:");
    scanf("%d",&n);
    for(i=1;i<=n;i++){
        fact=fact*i;
    }
    printf("\nFactorial is:%d",fact);

    return fact;
}
int main(){
    factorial();
}
```

9. Arrays in C

LAB EXERCISE:

- Write a C program that stores 5 integers in a one-dimensional array and prints them. Extend this to handle a two-dimensional array (3x3 matrix) and calculate the sum of all elements.

10. Pointers in C

LAB EXERCISE:

- Write a C program to demonstrate pointer usage. Use a pointer to modify the value of a variable and print the result.

11. Strings in C

LAB EXERCISE:

- Write a C program that takes two strings from the user and concatenates them using strcat(). Display the concatenated string and its length using strlen().

12. Structures in C

LAB EXERCISE:

- Write a C program that defines a structure to store a student's details (name, roll number, and marks). Use an array of structures to store details of 3 students and print them.

13. File Handling in C

LAB EXERCISE:

- Write a C program to create a file, write a string into it, close the file, then open the file again to read and display its contents.

Extra Lab Exercises for Improving Programming Logic

1. Operators

Lab Exercise 1: Simple Calculator

- Write a C program that acts as a simple calculator. The program should take two numbers and an operator as input from the user and perform the respective operation (addition, subtraction, multiplication, division, or modulus) using operators.
- Challenge: Extend the program to handle invalid operator inputs.

```
#include<stdio.h>
int main(){
    int i,j,k,choice;
    printf("\nPress 1 for addition.");
    printf("\nPress 2 for subtraction.");
    printf("\nPress 3 for multiplication.");
    printf("\nPress 4 for division.");
    printf("\nPress 5 for modulo.");
    printf("\n\nEnter your choice:");
    scanf("%d",&choice);

    switch(choice){
        case 1:
            printf("\nEnter any two value for calculation:");
            scanf("%d %d",&i,&j);
            k=i+j;
            printf("\nAddition:%d",k);
            break;

        case 2:
            printf("\nEnter any two value for calculation:");
            scanf("%d %d",&i,&j);
            k=i-j;
            printf("\nSubtraction:%d",k);
            break;
```

```

        case 3:
            printf("\nEnter any two value for calculation:");
            scanf("%d %d",&i,&j);
            k=i*j;
            printf("\nMultiplication:%d",k);
            break;

        case 4:
            printf("\nEnter any two value for calculation:");
            scanf("%d %d",&i,&j);
            k=i/j;
            printf("\nDivision:%d",k);
            break;

        case 5:
            printf("\nEnter any two value for calculation:");
            scanf("%d %d",&i,&j);
            k=i%j;
            printf("\nModulo:%d",k);
            break;

        default:
            printf("\nWrong input. Try again.");

    }
    return 0;
}

```

Lab Exercise 2: Check Number Properties

- Write a C program that takes an integer from the user and checks the following using different operators:
- Whether the number is even or odd.
- Whether the number is positive, negative, or zero.
- Whether the number is a multiple of both 3 and 5.

```

#include<stdio.h>
int main(){
    int i;
    printf("\nEnter a number:");
    scanf("%d",&i);

```



```

printf("\nThe number is%s", (i%2==0)? " even": " odd");
if(i>0){
    printf("\nThe number is positive.");
}
else if(i<0){
    printf("\nThe number is negative.");
}
else{
    printf("\nThe number is 0.");
}
printf("\nThe number is%s", (i%3==0 && i%5==0)? " divisible": " not
divisible");
}

```

2. Control Statements

Lab Exercise 1: Grade Calculator

- Write a C program that takes the marks of a student as input and displays the corresponding grade based on the following conditions:
- Marks > 90: Grade A
- Marks > 75 and <= 90: Grade B
- Marks > 50 and <= 75: Grade C
- Marks <= 50: Grade D
- Use if-else or switch statements for the decision-making process.

```

#include<stdio.h>
int main(){
    printf("\nMarks>90: Grade A");
    printf("\nMarks>75 and <=90: Grade B");
    printf("\nMarks>50 and <=75: Grade C");
    printf("\nMarks<=50: Grade D");

    float marks;
    printf("\n\nEnter your marks:");
    scanf("%f",&marks);

    if(marks>90){
        printf("\nYour grade is A.");
    }
    else if(marks>75 && marks<=90){
        printf("\nYour grade is B.");
    }
}

```

```

    }
    else if(marks>50 && marks<=75){
        printf("\nYour grade is C.");
    }
    else{
        printf("\nYour grade is D.");
    }

    return 0;
}

```

Lab Exercise 2: Number Comparison

- Write a C program that takes three numbers from the user and determines the largest number and the smallest number.
- Challenge: Solve the problem using both if-else and switch-case statements.

```

#include<stdio.h>
int main(){

    int a,b,c,choice;

    printf("\nPress 1 to find large numb or 2 for small number.");
    scanf("%d",&choice);

    printf("\nEnter any 3 numbers:");
    scanf("%d %d %d",&a,&b,&c);

    if(a>b && a>c){
        printf("\nA is the largest.");
    }
    else if(b>a && b>c){
        printf("\nB is the largest.");
    }
    else{
        printf("\nC is the largest.");
    }

    switch(choice){
        case 1:
            printf("\nEnter any 3 numbers:");
            scanf("%d %d %d",&a,&b,&c);
            a<b && a<c;

```

```

        printf("\nA is the smallest.");
        break;

    case 2:
        printf("\nEnter any 3 numbers:");
        scanf("%d %d %d",&a,&b,&c);
        b<a && b<c;
        printf("\nB is the smallest.");
        break;

    default:
        printf("\nC is the smallest.");

    }
}

```

3. Loops

Lab Exercise 1: Prime Number Check

- Write a C program that checks whether a given number is a prime number or not using a for loop.
- Challenge: Modify the program to print all prime numbers between 1 and a given number.

```

#include <stdio.h>

int main() {
    int n, i, Prime = 1;

    printf("Enter a number: ");
    scanf("%d", &n);

    if (n <= 1) {
        Prime = 0;
    } else {
        for (i = 2; i < n; i++) {
            if (n % i == 0) {
                Prime = 0;
                break;
            }
        }
    }
}

```

```

    }
}

if (isPrime)
    printf("%d is a PRIME number.\n", n);
else
    printf("%d is NOT a prime number.\n", n);

return 0;
}

```

Lab Exercise 2: Multiplication Table

- Write a C program that takes an integer input from the user and prints its multiplication table using a for loop.
- Challenge: Allow the user to input the range of the multiplication table (e.g., from 1 to N).

```

#include <stdio.h>

int main() {
    int n, i;

    printf("Enter a number: ");
    scanf("%d", &n);

    printf("\nMultiplication table of %d:\n", n);
    printf("-----\n");

    for (i = 1; i <= 10; i++) {
        printf("%d x %2d = %d\n", n, i, n * i);
    }

    return 0;
}

```

Lab Exercise 3: Sum of Digits

- Write a C program that takes an integer from the user and calculates the sum of its digits using a while loop.
- Challenge: Extend the program to reverse the digits of the number.

```

#include <stdio.h>

int main() {

```

```

int n, digit, sum = 0;

printf("Enter an integer: ");
scanf("%d", &n);

int temp = (n < 0) ? -n : n;

while (temp != 0) {
    digit = temp % 10;
    sum += digit;
    temp /= 10;
}

printf("Sum of digits of %d = %d\n", n, sum);
return 0;
}

```

4. Arrays

Lab Exercise 1: Maximum and Minimum in Array

- Write a C program that accepts 10 integers from the user and stores them in an array. The program should then find and print the maximum and minimum values in the array.
- Challenge: Extend the program to sort the array in ascending order.

```

#include <stdio.h>

int main() {
    int size = 10;
    int arr[10], i;
    int max, min, j, temp;
    printf("Enter %d integers:\n", size);
    for (i = 0; i < size; i++) {
        printf(" [%d]: ", i + 1);
        scanf("%d", &arr[i]);
    }

    max = arr[0];
    min = arr[0];

    for (i = 1; i < size; i++) {
        if (arr[i] > max) max = arr[i];
    }
}

```

```

        if (arr[i] < min) min = arr[i];
    }

    printf("\nMaximum value: %d\n", max);
    printf("Minimum value: %d\n", min);

    for (i = 0; i < size - 1; i++) {
        for (j = 0; j < size - 1 - i; j++) {
            if (arr[j] > arr[j + 1]) {
                temp    = arr[j];
                arr[j]  = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
}

printf("Sorted array : ");

for (i = 0; i < size; i++)
    printf("%d ", arr[i]);
printf("\n");
return 0;
}

```

Lab Exercise 2: Matrix Addition

- Write a C program that accepts two 2x2 matrices from the user and adds them. Display the resultant matrix.
- Challenge: Extend the program to work with 3x3 matrices and matrix multiplication.

```

#include <stdio.h>

void printMatrix2(int m[2][2]) {
    int i, j;
    for (i = 0; i < 2; i++) {
        printf(" | ");
        for (j = 0; j < 2; j++)
            printf("%4d ", m[i][j]);
        printf("\n");
    }
}

```

```

int main() {
    int A[2][2], B[2][2], C[2][2];
    int i, j;

    printf("Enter elements of Matrix A (2x2):\n");
    for (i = 0; i < 2; i++)
        for (j = 0; j < 2; j++) {
            printf(" A[%d][%d]: ", i, j);
            scanf("%d", &A[i][j]);
        }

    printf("Enter elements of Matrix B (2x2):\n");
    for (i = 0; i < 2; i++)
        for (j = 0; j < 2; j++) {
            printf(" B[%d][%d]: ", i, j);
            scanf("%d", &B[i][j]);
        }

    for (i = 0; i < 2; i++)
        for (j = 0; j < 2; j++)
            C[i][j] = A[i][j] + B[i][j];

    printf("\nMatrix A:\n"); printMatrix2(A);
    printf("Matrix B:\n"); printMatrix2(B);
    printf("A + B : \n"); printMatrix2(C);

    return 0;
}

```

Lab Exercise 3: Sum of Array Elements

- Write a C program that takes N numbers from the user and stores them in an array. The program should then calculate and display the sum of all array elements.
- Challenge: Modify the program to also find the average of the numbers.

```
#include <stdio.h>
```

```

int main() {
    int n, i;
    int sum = 0;

    printf("How many numbers? ");

```

```

scanf("%d", &n);

int arr[n];

printf("Enter %d numbers:\n", n);
for (i = 0; i < n; i++) {
    printf(" [%d]: ", i + 1);
    scanf("%d", &arr[i]);
}

for (i = 0; i < n; i++)
    sum += arr[i];

printf("\nSum of %d numbers = %d\n", n, sum);

return 0;
}

```

5. Functions

Lab Exercise 1: Fibonacci Sequence

- Write a C program that generates the Fibonacci sequence up to N terms using a recursive function.
- Challenge: Modify the program to calculate the Nth Fibonacci number using both iterative and recursive methods. Compare their efficiency.

```

#include <stdio.h>

int fibonacci(int n) {
    if (n == 0) return 0;
    if (n == 1) return 1;
    return fibonacci(n - 1) + fibonacci(n - 2);
}

int main() {
    int n, i;

    printf("Enter number of terms: ");
    scanf("%d", &n);

    printf("Fibonacci sequence (%d terms):\n", n);
    for (i = 0; i < n; i++)

```



```

        printf("%d ", fibonacci(i));

    printf("\n");
    return 0;
}

```

Lab Exercise 2: Factorial Calculation

- Write a C program that calculates the factorial of a given number using a function.
- Challenge: Implement both an iterative and a recursive version of the factorial function and compare their performance for large numbers.

```

#include <stdio.h>

int findFactorial(int n) {
    int fact = 1;

    for (int i = 1; i <= n; i++) {
        fact = fact * i;
    }

    return fact;
}

int main() {
    int num, result;

    printf("Enter a number: ");
    scanf("%d", &num);

    result = findFactorial(num);

    printf("Factorial of %d is %d\n", num, result);

    return 0;
}

```

Lab Exercise 3: Palindrome Check

- Write a C program that takes a number as input and checks whether it is a palindrome using a function.
- Challenge: Modify the program to check if a given string is a palindrome.

```

#include<stdio.h>

```

```

int isNumberPalindrome(int num) {
    int original = num;
    int reversed = 0;
    int remainder;

    while (num > 0) {
        remainder = num % 10;
        reversed = reversed * 10 + remainder;
        num = num / 10;
    }

    if (original == reversed) {
        return 1;
    } else {
        return 0;
    }
}

int main() {
    int number;

    printf("Enter an integer: ");
    scanf("%d", &number);

    if (isNumberPalindrome(number)) {
        printf("%d is a palindrome.\n", number);
    } else {
        printf("%d is not a palindrome.\n", number);
    }

    return 0;
}

```

6. Strings

Lab Exercise 1: String Reversal

- Write a C program that takes a string as input and reverses it using a function.
- Challenge: Write the program without using built-in string handling functions.

```
#include <stdio.h>
```

```

void reverseString(char str[]) {
    int length = 0;

    while (str[length] != '\0') {
        length++;
    }

    int start = 0;
    int end = length - 1;
    char temp;

    while (start < end) {
        temp = str[start];
        str[start] = str[end];
        str[end] = temp;

        start++;
        end--;
    }
}

int main() {
    char word[100];

    printf("Enter a string: ");
    scanf("%s", word);

    reverseString(word);

    printf("Reversed string: %s\n", word);

    return 0;
}

```

Lab Exercise 2: Count Vowels and Consonants

- Write a C program that takes a string from the user and counts the number of vowels and consonants in the string.
- Challenge: Extend the program to also count digits and special characters.

Lab Exercise 3: Word Count

- Write a C program that counts the number of words in a sentence entered by the user.
- Challenge: Modify the program to find the longest word in the sentence.

Extra Logic Building Challenges

Lab Challenge 1: Armstrong Number

- Write a C program that checks whether a given number is an Armstrong number or not (e.g., $153 = 1^3 + 5^3 + 3^3$).
- Challenge: Write a program to find all Armstrong numbers between 1 and 1000.

Lab Challenge 2: Pascal's Triangle

- Write a C program that generates Pascal's Triangle up to N rows using loops.
- Challenge: Implement the same program using a recursive function.

```
#include <stdio.h>

void print(int rows) {
    for (int i = 0; i < rows; i++) {

        for (int space = 1; space <= rows - i; space++) {
            printf(" ");
        }

        int number = 1;

        for (int j = 0; j <= i; j++) {
            printf("%4d", number);

            number = number * (i - j) / (j + 1);
        }

        printf("\n");
    }
}

int main()
{
    int n;

    printf("Enter the number of rows: ");
    scanf("%d", &n);

    print(n);
}
```

```
    return 0;
}
```

Lab Challenge 3: Number Guessing Game

- Write a C program that implements a simple number guessing game. The program should generate a random number between 1 and 100, and the user should guess the number within a limited number of attempts.
- Challenge: Provide hints to the user if the guessed number is too high or too low.

```
#include <stdio.h>
#include <stdlib.h> // Needed for rand()

int main() {
    int i, j;
    int max_levels = 3;

    printf("\nLets play a guessing game.");

    for (i = 1; i <= max_levels; i++) {
        printf("\n\nThis is level %d. Good luck", i);

        // Generates a random number between 1 and 10 without using time
        int magic_number = (rand() % 10) + 1;

        do {
            printf("\nGuess the magic number (1-10): ");
            scanf("%d", &j);

            if (j < magic_number) {
                printf("Uh-oh! Go Higher");
            }
            else if (j > magic_number) {
                printf("Uh-oh! Go Lower");
            }

        } while (j != magic_number);

        printf("\nCorrect! Level %d cleared.", i);
    }

    printf("\n\nCongrats! You win!\n");
}
```

```
    return 0;  
}
```